به نام خدا

گزارش کار آزمایش پنجم

ابراهیم صدیقی – علیرضا ندافی 9931098 – 9931090

گام اول: با اجرای کد داده شده، مشخصات پردازنده گرافیکی را بررسی می کنیم:

```
There is 1 device supporting CUDA

Device 0: "NVIDIA GeForce MX450"
  Major revision number:                         7
  Minor revision number:                         5
  Total amount of global memory:                 1769144320 bytes
  Number of multiprocessors:                     14
  Number of cores:                               112
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per block:           1024
  Maximum sizes of each dimension of a block:    1024 x 1024 x 64
  Maximum sizes of each dimension of a grid:     2147483647 x 65535 x 65535
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Clock rate:                                    1.58 GHz
  Concurrent copy and execution:                 Yes

TEST PASSED
```

همانطور که در تصویر مشخص است، یک عدد gpu روی دستگاه وجود دارد که مدل آن، و نسخه معماری آن (major و minor revision) قابل مشاهده است

تعداد SMها ۱۴ است که هر کدام نیز دارای ۸ هسته میباشند که تعداد کل هسته‌ها را شامل میشود.

اطلاعاتی از قبلی حافظه shared memory هر بلاک و تعداد رجیستر اختصاص یافته برای هر بلاک قابل ملاحظه است.

همچنین سایز وارپ که ۳۲ است به چشم میخورد.

تعداد نخ‌های هر بلاک برابر ۱۰۲۴ ذکر شده و حداکثر ابعاد بلاک و گرید نیز نوشته شده.

در آخر سایز حافظه اختصاصی و کلاک gpu نوشته شده.

گام دوم:

در این گام ابتدا متغیرهایی برای نگهداری آدرس شروع بردارها تعریف می شوند و سپس پردازنده گرافیکی که می خواهیم با آن عملیات را انجام دهیم انتخاب می کنیم. مطابق با دستورکار دستگاه 0 را انتخاب می کنیم. سپس باید از پردازنده گرافیکی برای آن سه بردار حافظه بگیریم و سپس دو بردار a و b که بردارهایی هستند که می خواهیم جمعشان کنیم را در حافظه پردازنده گرافیکی کپی می کنیم که بتوانیم عملیات را روی آنها انجام دهیم. اکنون نوبت به اجرای عمل جمع به صورت موازی در پردازنده گرافیکی می رسد. چون اندازه آرایه 1024 است، کرنلی که برای انجام عمل ضرب می نویسیم، هم 1024 نخ خواهد داشت و به همین دلیل تنها از x.threadIdx استفاده می کنیم و آن را در یک بلوک با 1024 نخ اجرا می کنیم. توجه داریم که باید cpu باید تا انتهای عمل gpu صبر کند، به همین دلیل باید آن را با gpu همگام کنیم. در انتها بعد از اطمینان از عدم وقوع خطا، نتیجه آرایه c را به cpu کپی می کنیم.

کد زیر پیاده سازی مان برای جمع برداری است:

```
/*
*       In His Exalted Name
*       Vector Addition - Sequential Code
*       Ahmad Siavashi, Email: siavashi@aut.ac.ir
*       21/05/2018
*/

#include <stdlib.h>
#include <stdio.h>
#include <time.h>

#include <cuda_runtime.h>
#include <device_launch_parameters.h>

#define BLOCK_SIZE 1024

// 0 -> one thread = one add
// 1 -> one thread = n add
#define MOD 0

void fillVector(int * v, size_t n);
void addVector(int * a, int *b, int *c, size_t n);
void printVector(int * v, size_t n);
cudaError_t addVectorGPU(const int* a, const int* b, int *c, size_t n, int num_blocks);

__global__ void addKernel1(int *c, const int *a, const int *b){
        int i = blockIdx.x * blockDim.x + threadIdx.x;
        c[i] = a[i] + b[i];
}

__global__ void addKernel2(int *c, const int *a, const int *b, size_t n){
        int i = threadIdx.x;
        int start = i * (n / BLOCK_SIZE), end = (i+1) * (n / BLOCK_SIZE);
        for(int j = start; j < end && j < n; j++)
                c[j] = a[j] + b[j];
}

cudaError_t addVectorGPU(const int* a, const int* b, int *c, size_t size, int num_blocks){
        int *dev_a, *dev_b, *dev_c;
        cudaError_t cudaStatus;

        cudaStatus = cudaSetDevice(0);
        if(cudaStatus != cudaSuccess) printf("CUDA Device failed!");

        cudaStatus = cudaMalloc((void**)&dev_c, size * sizeof(int));
        if(cudaStatus != cudaSuccess) printf("CudaMalloc failed for c!");

        cudaStatus = cudaMalloc((void**)&dev_a, size * sizeof(int));
        if(cudaStatus != cudaSuccess) printf("CudaMalloc failed for a!");

        cudaStatus = cudaMalloc((void**)&dev_b, size * sizeof(int));
        if(cudaStatus != cudaSuccess) printf("CudaMalloc failed for b!");

        cudaStatus = cudaMemcpy(dev_a, a, size * sizeof(int), cudaMemcpyHostToDevice);
        if(cudaStatus != cudaSuccess) printf("CudaMallocHostToDevice failed for a!");

        cudaStatus = cudaMemcpy(dev_b, b, size * sizeof(int), cudaMemcpyHostToDevice);
        if(cudaStatus != cudaSuccess) printf("CudaMallocHostToDevice failed for b!");

        if(MOD == 0){
                printf("Using addKernel1 ... \n");
                if(size % num_blocks != 0 || size/num_blocks > BLOCK_SIZE) {
                        printf("size, num_blocks not valid: %d, %d\n", (int)size, num_blocks);
                        return cudaStatus;
                }

                addKernel1 <<<num_blocks, size/num_blocks>>> (dev_c, dev_a, dev_b);
```

```
                addKernel1 <<<num_blocks, size/num_blocks>>> (dev_c, dev_a, dev_b);

        } else if(MOD == 1) {
                printf("Using addKernel2 ... \n");
                addKernel2 <<<1, BLOCK_SIZE>>> (dev_c, dev_a, dev_b, size);
        }

    cudaStatus = cudaGetLastError();
    if(cudaStatus ≠ cudaSuccess) printf("addKernel failed: %s\n", cudaGetErrorString(cudaStatus));

    cudaStatus = cudaDeviceSynchronize();
    if(cudaStatus ≠ cudaSuccess) printf("addKernel sync failed: %s\n", cudaGetErrorString(cudaStatus));

        cudaStatus = cudaMemcpy(c, dev_c, size * sizeof(int), cudaMemcpyDeviceToHost);
        if(cudaStatus ≠ cudaSuccess) printf("CudaMallocDeviceToHost failed for c!");

        cudaFree(dev_c);
        cudaFree(dev_b);
        cudaFree(dev_a);

        return cudaStatus;
}

int main()
{
        const int vectorSize = 102400, num_blocks = 100; // num_blocks is used for mod=0 only
        int a[vectorSize], b[vectorSize], c[vectorSize];

        fillVector(a, vectorSize);
        fillVector(b, vectorSize);

        // printf("Calling serial ... \n\n");
        // addVector(a, b, c, vectorSize);

        //printVector(c, vectorSize);

        printf("Calling GPU ... \n\n");
        addVectorGPU(a, b, c, vectorSize, num_blocks);

        printVector(c, vectorSize);


        return EXIT_SUCCESS;
}

// Fills a vector with data
void fillVector(int * v, size_t n) {
        int i;
        for (i = 0; i < n; i++) {
                v[i] = i;
        }
}

// Adds two vectors
void addVector(int * a, int *b, int *c, size_t n) {
        int i;
        for (i = 0; i < n; i++) {
                c[i] = a[i] + b[i];
        }
}
```

```
// Prints a vector to the stdout.
void printVector(int * v, size_t n) {
        int i;
        printf("[-] Vector elements: ");
        for (i = 0; i < n; i++) {
                printf("%d, ", v[i]);
        }
        printf("\b\b  \n");
}
```

وقتی MOD=0 باشد، به ازای هر عمل جمع یک ترد ساخته میشود که وظیفه‌اش انجام یک جمع است
وقتی MOD=1 باشد، به اندازه block size ترد ساخته میشود که هر ترد وظیفه اش انجام n / block_Size جمع است

گام سوم:
نحوه پیاده سازی برای گام سوم به این صورت است که ابتدا دو ثابت برای تعداد بالک و تعداد وظایفی که هر نخ انجام می دهد تعریف می کنیم. در صورتی که بخواهیم از روش اول)هر نخ n جمع انجام بدهد(گام سوم را انجام بدهیم، تعداد بالک را برابر 1 و تعداد وظیفه ای که هر نخ انجام می دهد را برابر n می گذاریم. در حالتی که بخواهیم n بالک 1024تایی اجرا کنیم، تعداد بالک را برابر n و تعداد وظیفه هر نخ را 1 می گذاریم
زمان اجرا در اجراهای متفاوت تفاوت قابل توجه داشت، اما علت آن را متوجه نشدیم. ولی به طور میانگین حدود ۰.۲ ثانیه بود

```
alireza@alireza-ThinkBook:~/Documents/lab5$ ./add0
Calling GPU...

Time taken for MOD=0 was 0.282066
alireza@alireza-ThinkBook:~/Documents/lab5$ ./add1
Calling GPU...

Time taken for MOD=1 was 0.228976
```

گام چهارم:
برای اجرای گام چهارم، به جهت اینکه بتوانیم در gpu از آرایه های cpu استفاده کنیم، از cudaMallocManaged() استفاده کردیم. سپس در کدی که برای کرنل نوشتیم، هر نخ شماره بالک خود، آیدی و وارپ مربوط به خود را در آرایه های متناظر با آن می نویسد. سپس در cpu مقادیر آن آرایه چاپ می شود. کد این گام در انتهای کد پیوست شده موجود است. نتایج به صورت زیر است:

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void ThreadDetails(int *block, int *warp, int *local_index) {
    int block_id = blockIdx.x;
    int thread_id = threadIdx.x;
    int thread = block_id * blockDim.x + thread_id;
    int warp_id = thread_id / warpSize;

    block[thread] = block_id;
    warp[thread] = warp_id;
    local_index[thread] = thread_id;
}

int main(int argc, char **argv) {
    int size = 64 * 2;

    int *h_block = (int *)malloc(size * sizeof(int));
    int *h_warp = (int *)malloc(size * sizeof(int));
    int *h_local_index = (int *)malloc(size * sizeof(int));

    int *d_block, *d_warp, *d_local_index;
    cudaMalloc((void **)&d_block, size * sizeof(int));
    cudaMalloc((void **)&d_warp, size * sizeof(int));
    cudaMalloc((void **)&d_local_index, size * sizeof(int));

    ThreadDetails<<<2, 64>>>(d_block, d_warp, d_local_index);
    cudaDeviceSynchronize();

    cudaMemcpy(h_block, d_block, size * sizeof(int), cudaMemcpyDeviceToHost);
    cudaMemcpy(h_warp, d_warp, size * sizeof(int), cudaMemcpyDeviceToHost);
    cudaMemcpy(h_local_index, d_local_index, size * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < size; i++) {
        printf("Calculated Thread: %d,\tBlock: %d,\tWarp %d,\tThread %d\n", i, h_block[i], h_warp[i], h_local_index[i]);
    }

    free(h_block);
    free(h_warp);
    free(h_local_index);
    cudaFree(d_block);
    cudaFree(d_warp);
    cudaFree(d_local_index);

    return 0;
}
```

```
Calculated Thread: 0,   Block: 0,       Warp 0, Thread 0
Calculated Thread: 1,   Block: 0,       Warp 0, Thread 1
Calculated Thread: 2,   Block: 0,       Warp 0, Thread 2
Calculated Thread: 3,   Block: 0,       Warp 0, Thread 3
Calculated Thread: 4,   Block: 0,       Warp 0, Thread 4
Calculated Thread: 5,   Block: 0,       Warp 0, Thread 5
Calculated Thread: 6,   Block: 0,       Warp 0, Thread 6
Calculated Thread: 7,   Block: 0,       Warp 0, Thread 7
Calculated Thread: 8,   Block: 0,       Warp 0, Thread 8
Calculated Thread: 9,   Block: 0,       Warp 0, Thread 9
Calculated Thread: 10,  Block: 0,       Warp 0, Thread 10
Calculated Thread: 11,  Block: 0,       Warp 0, Thread 11
Calculated Thread: 12,  Block: 0,       Warp 0, Thread 12
Calculated Thread: 13,  Block: 0,       Warp 0, Thread 13
Calculated Thread: 14,  Block: 0,       Warp 0, Thread 14
Calculated Thread: 15,  Block: 0,       Warp 0, Thread 15
Calculated Thread: 16,  Block: 0,       Warp 0, Thread 16
Calculated Thread: 17,  Block: 0,       Warp 0, Thread 17
Calculated Thread: 18,  Block: 0,       Warp 0, Thread 18
Calculated Thread: 19,  Block: 0,       Warp 0, Thread 19
Calculated Thread: 20,  Block: 0,       Warp 0, Thread 20
Calculated Thread: 21,  Block: 0,       Warp 0, Thread 21
Calculated Thread: 22,  Block: 0,       Warp 0, Thread 22
Calculated Thread: 23,  Block: 0,       Warp 0, Thread 23
Calculated Thread: 24,  Block: 0,       Warp 0, Thread 24
Calculated Thread: 25,  Block: 0,       Warp 0, Thread 25
Calculated Thread: 26,  Block: 0,       Warp 0, Thread 26
Calculated Thread: 27,  Block: 0,       Warp 0, Thread 27
Calculated Thread: 28,  Block: 0,       Warp 0, Thread 28
Calculated Thread: 29,  Block: 0,       Warp 0, Thread 29
Calculated Thread: 30,  Block: 0,       Warp 0, Thread 30
Calculated Thread: 31,  Block: 0,       Warp 0, Thread 31
Calculated Thread: 32,  Block: 0,       Warp 1, Thread 32
Calculated Thread: 33,  Block: 0,       Warp 1, Thread 33
Calculated Thread: 34,  Block: 0,       Warp 1, Thread 34
Calculated Thread: 35,  Block: 0,       Warp 1, Thread 35
Calculated Thread: 36,  Block: 0,       Warp 1, Thread 36
Calculated Thread: 37,  Block: 0,       Warp 1, Thread 37
Calculated Thread: 38,  Block: 0,       Warp 1, Thread 38
Calculated Thread: 39,  Block: 0,       Warp 1, Thread 39
Calculated Thread: 40,  Block: 0,       Warp 1, Thread 40
Calculated Thread: 41,  Block: 0,       Warp 1, Thread 41
Calculated Thread: 42,  Block: 0,       Warp 1, Thread 42
Calculated Thread: 43,  Block: 0,       Warp 1, Thread 43
Calculated Thread: 44,  Block: 0,       Warp 1, Thread 44
Calculated Thread: 45,  Block: 0,       Warp 1, Thread 45
Calculated Thread: 46,  Block: 0,       Warp 1, Thread 46
Calculated Thread: 47,  Block: 0,       Warp 1, Thread 47
Calculated Thread: 48,  Block: 0,       Warp 1, Thread 48
Calculated Thread: 49,  Block: 0,       Warp 1, Thread 49
Calculated Thread: 50,  Block: 0,       Warp 1, Thread 50
Calculated Thread: 51,  Block: 0,       Warp 1, Thread 51
Calculated Thread: 52,  Block: 0,       Warp 1, Thread 52
Calculated Thread: 53,  Block: 0,       Warp 1, Thread 53
Calculated Thread: 54,  Block: 0,       Warp 1, Thread 54
Calculated Thread: 55,  Block: 0,       Warp 1, Thread 55
Calculated Thread: 56,  Block: 0,       Warp 1, Thread 56
Calculated Thread: 57,  Block: 0,       Warp 1, Thread 57
Calculated Thread: 58,  Block: 0,       Warp 1, Thread 58
Calculated Thread: 59,  Block: 0,       Warp 1, Thread 59
Calculated Thread: 60,  Block: 0,       Warp 1, Thread 60
Calculated Thread: 61,  Block: 0,       Warp 1, Thread 61
Calculated Thread: 62,  Block: 0,       Warp 1, Thread 62
Calculated Thread: 63,  Block: 0,       Warp 1, Thread 63
Calculated Thread: 64,  Block: 1,       Warp 0, Thread 0
Calculated Thread: 65,  Block: 1,       Warp 0, Thread 1
Calculated Thread: 66,  Block: 1,       Warp 0, Thread 2
Calculated Thread: 67,  Block: 1,       Warp 0, Thread 3
```

```
Calculated Thread: 63,  Block: 0,       Warp 1, Thread 63
Calculated Thread: 64,  Block: 1,       Warp 0, Thread 0
Calculated Thread: 65,  Block: 1,       Warp 0, Thread 1
Calculated Thread: 66,  Block: 1,       Warp 0, Thread 2
Calculated Thread: 67,  Block: 1,       Warp 0, Thread 3
Calculated Thread: 68,  Block: 1,       Warp 0, Thread 4
Calculated Thread: 69,  Block: 1,       Warp 0, Thread 5
Calculated Thread: 70,  Block: 1,       Warp 0, Thread 6
Calculated Thread: 71,  Block: 1,       Warp 0, Thread 7
Calculated Thread: 72,  Block: 1,       Warp 0, Thread 8
Calculated Thread: 73,  Block: 1,       Warp 0, Thread 9
Calculated Thread: 74,  Block: 1,       Warp 0, Thread 10
Calculated Thread: 75,  Block: 1,       Warp 0, Thread 11
Calculated Thread: 76,  Block: 1,       Warp 0, Thread 12
Calculated Thread: 77,  Block: 1,       Warp 0, Thread 13
Calculated Thread: 78,  Block: 1,       Warp 0, Thread 14
Calculated Thread: 79,  Block: 1,       Warp 0, Thread 15
Calculated Thread: 80,  Block: 1,       Warp 0, Thread 16
Calculated Thread: 81,  Block: 1,       Warp 0, Thread 17
Calculated Thread: 82,  Block: 1,       Warp 0, Thread 18
Calculated Thread: 83,  Block: 1,       Warp 0, Thread 19
Calculated Thread: 84,  Block: 1,       Warp 0, Thread 20
Calculated Thread: 85,  Block: 1,       Warp 0, Thread 21
Calculated Thread: 86,  Block: 1,       Warp 0, Thread 22
Calculated Thread: 87,  Block: 1,       Warp 0, Thread 23
Calculated Thread: 88,  Block: 1,       Warp 0, Thread 24
Calculated Thread: 89,  Block: 1,       Warp 0, Thread 25
Calculated Thread: 90,  Block: 1,       Warp 0, Thread 26
Calculated Thread: 91,  Block: 1,       Warp 0, Thread 27
Calculated Thread: 92,  Block: 1,       Warp 0, Thread 28
Calculated Thread: 93,  Block: 1,       Warp 0, Thread 29
Calculated Thread: 94,  Block: 1,       Warp 0, Thread 30
Calculated Thread: 95,  Block: 1,       Warp 0, Thread 31
Calculated Thread: 96,  Block: 1,       Warp 1, Thread 32
Calculated Thread: 97,  Block: 1,       Warp 1, Thread 33
Calculated Thread: 98,  Block: 1,       Warp 1, Thread 34
Calculated Thread: 99,  Block: 1,       Warp 1, Thread 35
Calculated Thread: 100, Block: 1,       Warp 1, Thread 36
Calculated Thread: 101, Block: 1,       Warp 1, Thread 37
Calculated Thread: 102, Block: 1,       Warp 1, Thread 38
Calculated Thread: 103, Block: 1,       Warp 1, Thread 39
Calculated Thread: 104, Block: 1,       Warp 1, Thread 40
Calculated Thread: 105, Block: 1,       Warp 1, Thread 41
Calculated Thread: 106, Block: 1,       Warp 1, Thread 42
Calculated Thread: 107, Block: 1,       Warp 1, Thread 43
Calculated Thread: 108, Block: 1,       Warp 1, Thread 44
Calculated Thread: 109, Block: 1,       Warp 1, Thread 45
Calculated Thread: 110, Block: 1,       Warp 1, Thread 46
Calculated Thread: 111, Block: 1,       Warp 1, Thread 47
Calculated Thread: 112, Block: 1,       Warp 1, Thread 48
Calculated Thread: 113, Block: 1,       Warp 1, Thread 49
Calculated Thread: 114, Block: 1,       Warp 1, Thread 50
Calculated Thread: 115, Block: 1,       Warp 1, Thread 51
Calculated Thread: 116, Block: 1,       Warp 1, Thread 52
Calculated Thread: 117, Block: 1,       Warp 1, Thread 53
Calculated Thread: 118, Block: 1,       Warp 1, Thread 54
Calculated Thread: 119, Block: 1,       Warp 1, Thread 55
Calculated Thread: 120, Block: 1,       Warp 1, Thread 56
Calculated Thread: 121, Block: 1,       Warp 1, Thread 57
Calculated Thread: 122, Block: 1,       Warp 1, Thread 58
Calculated Thread: 123, Block: 1,       Warp 1, Thread 59
Calculated Thread: 124, Block: 1,       Warp 1, Thread 60
Calculated Thread: 125, Block: 1,       Warp 1, Thread 61
Calculated Thread: 126, Block: 1,       Warp 1, Thread 62
Calculated Thread: 127, Block: 1,       Warp 1, Thread 63
```